

DIO: Black-Box Online Routing for Heterogeneous vLLM Replicas

Keyush Nisar¹, Krisha Maisheri¹, Krishil Parikh¹,
Aruna Gawade¹, Nilesh T. Rathod¹, Angelin Florence A¹

¹Department of Artificial Intelligence and Machine Learning,
Dwarkadas J. Sanghvi College of Engineering, Mumbai, India.

Contributing authors: keyush.nisar@djsce.edu.in;
krisha.maisheri@djsce.edu.in; krishil.parikh@djsce.edu.in;
aruna.gawade@djsce.ac.in; nilesh.rathod@djsce.ac.in;
angelin.florence@djsce.ac.in;

Abstract

Multi-instance vLLM deployments often use Round-Robin or least-connections even when replica cost diverges. We present **DIO** (**dio-serve**; system name, not an acronym), a thin OpenAI-compatible gateway that ranks backends from black-box end-to-end telemetry with a dual-timescale normalized LMS-style model, without engine patches or an offline latency/throughput profile. On matched dual Tesla T4 replicas, ranking stays near Round-Robin. Under controlled $\times 2$ end-to-end latency inflation, mean p99 improves by **48.3%** (SD **0.7%**) over Round-Robin ($n=10$). On physical T4+A30, DIO routes **89.5%** of mixed-length traffic to the empirically lower-latency A30 and improves mean p99 by **33.3%** (SD **17.4%**) across five paired seeds. Absolute admission on the same model rejects **0.40 ± 0.21** of dual-T4 requests, so ranking and admission must be decoupled. Code and artifacts are public.

Keywords: LLM serving, online routing, heterogeneous GPU clusters, vLLM, latency ranking, NLMS.

Corresponding author: Keyush Nisar, Department of Artificial Intelligence and Machine Learning, Dwarkadas J. Sanghvi College of Engineering, Mumbai, India. E-mail: keyush.nisar@djsce.edu.in.

1 Introduction

Multi-instance vLLM [1] (or SGLang/TGI [2–4]) deployments commonly place an L7 gateway such as Nginx [5], Kubernetes Services [6], Envoy [7], or Ray Serve [8] in front of independent inference replicas with Round-Robin or least-connections. Single-node engines optimize continuous batching [9, 10]; they do not solve *which backend* should take the next request. Simple L7 policies such as Round-Robin and least-connections [5, 11] treat backends as interchangeable capacity units.

In production, effective cost still diverges even on identical GPU types: pending queues grow unevenly, KV-cache utilization spikes on one replica, prefix caches favor sessions pinned to a peer, and host interference can temporarily slow one process. These differences can lead to avoidable tail latency [12]. Co-designed cluster schedulers [13–15] can react with deep engine hooks or live KV migration, but many operators need a non-invasive front door that wraps unmodified OpenAI-compatible [16] inference APIs without patches.

A second problem arises when the same online latency model is used for admission control. Models learned from black-box end-to-end telemetry are often accurate only as *relative rankers*. On dual Tesla T4 we measure dual-timescale NLMS [17] MAPE of $\sim 90\text{--}130\%$. Gating admission on “reject if $\min \hat{y} > \text{SLO}$ ” inherits that error and can starve healthy traffic.

We design **DIO** (**dio-serve**) as a thin control plane for this multi-instance pattern (Figure 1; hybrid scrape, affinity, and admission are optional paths—not the central experimental claim). Its core loop uses request telemetry (end-to-end latency, tokens, and gateway pending) to rank backends online. When available, DIO can additionally consume engine metrics from each vLLM Prometheus endpoint [18]. The gateway is HTTP-only and leaves the engine source and runtime unmodified. Optional session affinity and admission modes are reported as operational capabilities rather than as the central performance claim.

Empirically, DIO remains near Round-Robin parity on homogeneous dual-T4 peers, responds to a controlled end-to-end latency gap, and routes mixed-length traffic to the empirically lower-latency GPU in a T4+A30 pair without requiring an offline latency/throughput performance-capacity model before deployment (Section 7.5). We separately report the admission failure mode caused by treating a relative ranker as an absolute SLO predictor.

Contributions.

1. **Online routing from black-box telemetry.** DIO uses dual-timescale normalized LMS-style ranking with a constant-time update for the two-parameter model (avoiding matrix operations of generic $O(d^2)$ RLS) to rank unmodified vLLM workers from end-to-end latency and token counts under an OpenAI-compatible HTTP boundary, without requiring an offline latency/throughput model before deployment.
2. **Evaluation on physically heterogeneous GPUs.** On real T4+A30 unmodified vLLM (no injected delay), dual-timescale NLMS matches $d=2$ RLS on mean p99 while showing lower routing-fraction variability toward the A30 (89.5% mean frac) and improves mean p99 by 33.3% (SD 17.4%) over Round-Robin in five paired seeds.

3. **Admission calibration analysis.** Homogeneous parity and a controlled slow-peer experiment frame the ranking claim; we show that a ranker with high absolute error must not be used directly as an SLO threshold.

Implementation, scripts, and multi-seed artifacts for the dual-T4 and T4+A30 experiments are released with the paper (Declarations).

2 Background and related work

2.1 Single-node engines and prefix/KV systems

vLLM [1] popularized PagedAttention and continuous batching; alternatives revisit memory management without paging [19]. SGLang [2, 4] improves structured decoding and RadixAttention prefix reuse. TGI [3] targets production APIs. Hybrid iteration policies include Sarathi-Serve [10] and FastServe [20]. Disaggregated prefill/decode systems [21–24] optimize pipeline stages and KV placement. Prefix/KV reuse [25–29] raises the value of *where* a request lands. These systems excel on one node or a co-designed cluster; multi-worker routing in front of *unmodified* engines remains external.

2.2 Cluster orchestrators and classical serving

Ray Serve [8] and Triton [30] emphasize actors and model repositories with coarse metrics. Earlier DNN serving stacks (Clipper [31], INFaaS [32], Clockwork [33]) established predictability and model-selection themes that LLM gateways inherit. Orca [9] advances iteration-level batching. AlpaServe [34] and CoCoServe [35] focus on placement and multiplexing. Mélange [36] targets heterogeneous capacity tables. NexusSched [13] combines predictive routing with engine co-design and multi-feature predictors ($O(d^2)$ updates). Llumnix [14] migrates live KV state. LoongServe [15] targets long-context elastic parallelism. ServerlessLLM [37] optimizes cold-start loading.

Inference-aware gateways. Metrics-, prefix-, and KV-aware routing over unmodified vLLM is already deployed infrastructure. The Kubernetes Gateway API Inference Extension [38] and its Endpoint Picker select replicas using scraped vLLM queue depth, KV utilization, and LoRA state; llm-d [39] and Google’s GKE Inference Gateway [40] build on it. NVIDIA Dynamo [41] scores endpoints by cached-prefix overlap. The vLLM production-stack router [42] ships round-robin, session, prefix-aware, and KV-aware policies (including static backends without Kubernetes). Alibaba’s ACK/ACS gateway [43], the SGLang router [44], and Envoy AI Gateway [45] similarly offer cache- or LLM-aware balancing. These systems establish that engine-metric and locality-aware routing are practical.

Focus of this work. We study whether a lightweight gateway can learn relative per-replica service rates from black-box end-to-end telemetry without engine modification or an offline latency/throughput profile, and when such a model is unsafe for absolute admission. The routing signal is dual-timescale NLMS-style ranking [17] (Section 3.2) with a constant-time update for the two-parameter model. Closest learned comparators either need engine co-design or generic $O(d^2)$ RLS [13]. On physical multi-SKU load, NLMS and RLS achieve similar mean p99, with NLMS showing lower routing-fraction variability (Section 7.5). The same model is poorly calibrated in absolute terms (MAPE $\sim 90\text{--}130\%$, Section 7.3), so a textbook absolute

SLO gate rejects $\sim 40\%$ of a load that empirical and rank-only gating complete in full (Section 7.6). Section 8 summarizes deployment trade-offs relative to production stacks.

2.3 Adaptive prediction, queues, and admission

We compare NLMS with two complementary baselines: RLS, which uses the same linear feature model but higher update cost [17, 46], and Latency-EWMA, a simpler ranker of recent end-to-end latency that ignores token count (Section 7.3). Queue wait uses a batch-amortized Little-style heuristic [47, 48]. SJF-style ranking motivates predicted service routing [49]. Roofline [50] inspires soft memory ceilings recast as routing penalties. Goodput- and SLO-aware LLM serving [21, 51] motivates treating admission as a separate policy from ranking. Simulation frameworks such as Vidur [52] characterize offline capacity; DIO targets online multi-instance ranking under weak absolute models.

3 System design

3.1 Architecture and software artifact

Figure 1 shows **dio-serve**, the gateway used in all experiments below. Clients issue OpenAI-compatible HTTP requests. A FastAPI control plane ranks backends with dual-timescale NLMS, optionally folds in scraped engine metrics, applies session affinity and admission, then proxies to an unmodified vLLM worker. Completions update per-worker models from observed e2e latency and token counts. A Go/gRPC research path exists in the repository but is not used for the results in this paper.

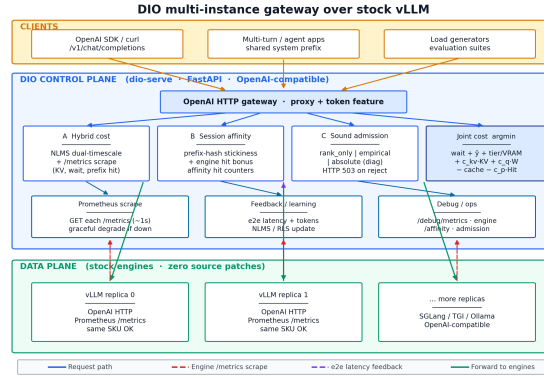


Fig. 1 DIO multi-instance gateway. Clients call **dio-serve**; the control plane ranks unmodified vLLM replicas with NLMS, optional Prometheus scrape, session affinity, and admission (no engine patches).

Telemetry uses four signals: gateway e2e latency; token counts from response **usage** or a tokenizer estimate; per-backend pending depth; and optional HTTP scrape of each engine **/metrics**. Soft and hard VRAM limits come from configuration or KV usage. Unlike NexusSched and Llumnix, which require engine hooks or live migration [13, 14], DIO does not sample NVML SM/power/temperature and does not modify engine source.

A typical launch is `python -m dio serve -b e0=URL0 -b e1=URL1` (package name `dio-serve`). Background tasks scrape `/metrics` every 1s by default and probe health. Scheduling overhead is on the order of 10–15 μ s per request in a local microbench (Section 7.2). Source code and experiment scripts are public (Declarations).

3.2 Dual-timescale normalized LMS-style latency model

For worker w and token count N we use the linear model

$$\hat{y}_w = s_w N + b_w, \quad (1)$$

with slope s_w (ms/token) and intercept b_w (ms), i.e. feature vector $x=[N, 1]$. After a completion with residual $e = y - \hat{y}$, we apply a *dual-timescale normalized LMS-style* update (not textbook multi-feature NLMS with $\|x\|^2$ normalization):

$$s \leftarrow s + \mu \frac{e}{N}, \quad b \leftarrow b + \mu_b e, \quad (2)$$

where the slope step is scaled by $1/N$ (scalar normalization for the token feature) and the intercept uses a separate fixed step μ_b . We keep two timescales for the slope,

$$s^{\text{eff}} = \alpha s^{\text{fast}} + (1 - \alpha) s^{\text{slow}}, \quad \alpha=0.8, \quad (3)$$

with $\mu_{\text{fast}}=0.1$, $\mu_{\text{slow}}=0.01$, and $\mu_b=0.005$. Priors are $s_0=2.0$ ms/token and $b_0=150$ ms. For this two-parameter model the update is constant-time (no matrix operations as in generic $O(d^2)$ RLS). Slopes typically leave the prior after about 15–20 completions per worker. Figure 2 shows a simulated slope jump. The token feature N prefers Hugging Face tokenizer counts plus planned `max_tokens`, with fallback `[|prompt|/4] + max_tokens`.

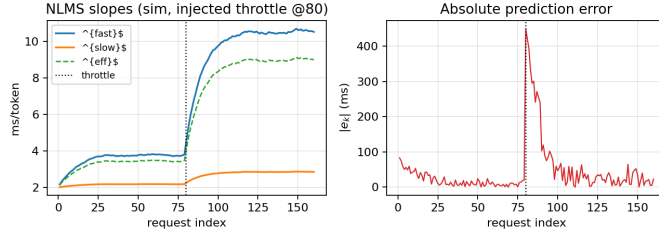


Fig. 2 Simulated dual-timescale normalized LMS-style slope under an injected jump at request 80. s^{fast} adapts first; s^{slow} lags. Not dual-T4 wall-clock traces.

3.3 Baseline joint cost

Absent hybrid metrics, each healthy worker scores

$$S_w^{\text{base}} = \text{wait}_w + \hat{t}_w + \text{tierCost}_{r,w} + \text{vramCost}_w - \text{cacheBonus}_w, \quad (4)$$

with $\hat{t}_w = s_w^{\text{eff}} N + b_w$, $\text{wait}_w = (Q_w/B)\bar{t}_w$ ($B=8$), soft VRAM penalty when free VRAM < 4 GB, and hard exclusion when free VRAM < 2.4 GB and $N > 1000$. Operators

may label each worker with an optional *tier* index (an integer capability class, e.g., small vs. large GPU or model size). Then $\text{tierCost}_{r,w}$ is a soft 500 ms penalty when the request’s declared size class exceeds the worker’s tier, and a hard block applies when a large request would land on a small-tier worker (“large-on-small”). In all dual-T4 and T4+A30 experiments in this paper, every worker shares the same tier, so $\text{tierCost}=0$ and hard tier blocks never fire; the term is retained for mixed-capability fleets. Route to $\arg \min_w S_w$ in $O(|W|)$ time. Strategies also include Round-Robin, Least-Loaded, static slopes, and $d=2$ RLS as head-to-head baselines under the *same* cost shell when applicable.

4 Hybrid cost routing with engine metrics

4.1 Why engine metrics

NLMS observes only gateway e2e latency and tokens. That signal is delayed, noisy, and confounded by batching. Unmodified vLLM already exports internal state via Prometheus [18]: GPU KV-cache utilization, waiting/running request counts, and prefix-cache hit counters. DIO optionally scrapes those gauges and folds them into the same millisecond-scale cost used for ranking, without source modification. We describe but do not experimentally evaluate the optional hybrid and affinity paths as independent performance claims in this submission; the primary ranking evidence is dual-timescale NLMS-style routing from black-box e2e telemetry (Sections 7.4–7.5).

4.2 Metric scrape and snapshot

For each backend i with base URL U_i and metrics path (default `/metrics`), a background task issues `GET U_i/metrics` and parses Prometheus exposition text. Metric name variants across vLLM versions are handled (e.g. `vllm:gpu_cache_usage_perc` vs. `vllm:kv_cache_usage_perc`; prefix hit rate from a direct gauge or from hits/queries counters). Each scrape yields a snapshot

$$\mathcal{E}_w = (\text{KV}_w, W_w, R_w, \text{Hit}_w, \text{ok}_w), \quad (5)$$

where $\text{KV}_w \in [0, 1]$, W_w and R_w are waiting/running request counts, $\text{Hit}_w \in [0, 1]$ is prefix-cache hit rate, and ok_w is false on HTTP/parse failure. On failure, hybrid terms contribute zero (graceful degrade to NLMS-only ranking). Scrape interval defaults to 1 s—fast enough to track KV pressure, slow enough not to contend with decode.

4.3 Hybrid joint cost

When ok_w is true, the full score is

$$\begin{aligned} S_w = & \text{wait}_w + \hat{t}_w + \text{tierCost}_{r,w} + \text{vramCost}_w \\ & + c_{kv} \text{KV}_w + c_q W_w - \text{cacheBonus}_w - c_p \text{Hit}_w. \end{aligned} \quad (6)$$

Default hybrid coefficients (same ms units as other terms): $c_{kv}=800$, $c_q=50$ per waiting request, $c_p=150$. We selected these heuristic defaults to keep the hybrid terms on the same approximate millisecond scale as the base cost terms; they are configuration defaults, not parameters optimized by the physical-heterogeneity experiment. Operators may override all three through `DIO_*` settings.

If $KV_w > 0.95$ and $N > 500$, worker w is hard-blocked for that request (engine-reported near-full KV). Free-VRAM estimates may be updated as $free_w \approx total_w(1 - KV_w)$ when totals are known. When backends differ only in learned service slope and metrics look similar, NLMS ranking still dominates.

Algorithm 1 Worker selection with hybrid scrape, affinity, and admission

Require: request r , workers W , mode m , optional snapshots $\{\mathcal{E}_w\}$

```

1: Background (async): periodically scrape each backend /metrics into  $\mathcal{E}_w$  (KV,  $W$ , Hit); on failure leave hybrid terms at 0
2:  $N \leftarrow \text{TokenFeature}(r)$ ;  $w^* \leftarrow \perp$ ;  $S_{\min} \leftarrow \infty$ 
3: for all  $w \in W$  healthy and tier-feasible do
4:    $\hat{t}_w \leftarrow \text{Predict}(w, N)$ 
5:   if VRAM/KV hard-blocked( $w, N, \mathcal{E}_w$ ) then continue
6:   end if
7:    $S_w \leftarrow \text{wait} + \hat{t}_w + \text{tier} + \text{vram} + c_{kv}KV_w + c_qW_w$ 
    $\quad - \text{cacheBonus}(r, w) - c_p\text{Hit}_w$ 
8:   if  $S_w < S_{\min}$  then  $S_{\min} \leftarrow S_w$ ;  $w^* \leftarrow w$ 
9:   end if
10: end for
11: if  $w^* = \perp$  or AdmissionReject( $S_{\min}, m$ ) then
12:   return HTTP 503 with Retry-After
13: end if
14: RecordPrefixAffinity( $r, w^*$ ); return  $w^*$ 

```

5 Session and prefix-cache affinity

5.1 Why multi-turn affinity

Round-Robin and least-connections scatter multi-turn sessions across replicas. Each turn that lands on a cold replica re-prefills shared system prompts and agent context, missing in-engine prefix caches [4, 25, 26]. Cache-aware and prompt-scheduling systems [27, 29] show large gains when locality is preserved, but they often assume co-designed clusters. DIO targets several unmodified replicas behind one OpenAI-compatible gateway for multi-turn or agentic clients [53, 54], without engine patches.

5.2 Affinity mechanism

On each admitted request with prompt text p , DIO computes a stable prefix hash over the first 256 characters:

$$h(p) = \text{hash}(p[0:256]) \bmod 2^{32}. \quad (7)$$

A map $M : h \mapsto w$ records the last worker that successfully served that prefix. When scoring worker w , if $M[h(p)] = w$, a cache affinity bonus $\text{cacheBonus}_w = C$ (default $C=200$ ms) is subtracted from S_w . Independently, the hybrid term $c_p\text{Hit}_w$ prefers workers the *engine* reports as currently caching well. After admission, DIO sets $M[h(p)] \leftarrow w^*$.

DIO maintains `affinity_decisions` and `affinity_hits` (true when the chosen worker already owned the prefix). Exposed hit rate `affinity_hits/affinity_decisions` is the primary stickiness metric for multi-turn evaluation; operators can also compare engine-reported prefix hit rates under DIO vs. RR when live metrics are available. Affinity is request-level stickiness complementary to RadixAttention [4]: DIO increases the chance that the engine’s own cache is warm; it does not implement a second cache. The `no_cache` ablation disables the bonus for sensitivity studies.

6 Admission control under poor absolute calibration

6.1 Why decouple admission from absolute $\hat{y}$

Many predictive schedulers implicitly assume $\hat{y}$ is calibrated enough to compare against an SLO in absolute milliseconds. Section 7.3 shows that assumption fails for NLMS on dual-T4 (MAPE $\sim 90\text{--}130\%$). Cold-start priors and batching can make S_w larger than any reasonable SLO even when true latency is fine. Gating on $\min_w S_w > \text{SLO}$ then rejects healthy traffic—or, if priors are optimistic, admits work that will miss the SLO. Predictability-first serving [33] and goodput-oriented LLM systems [21, 51] motivate treating admission as a policy that must not inherit model bias.

6.2 Three admission modes

Ranking is always $\arg \min_w S_w$ among feasible workers. Admission is separate:

- **rank_only**: never reject for SLO based on S_w ; only hard VRAM/tier/KV blocks apply. NLMS is used only for ranking.
- **empirical** (default): after warm-up (≥ 8 samples in a window of length 64), reject only if the empirical p -th percentile (default $p=95$) exceeds the SLO *and* the best score lies in the worst quartile of feasible workers. Cold-start does not reject on $\hat{y}$ alone.
- **absolute** (diagnostic): reject if $\min_w S_w > \text{SLO}$. Sensitive to MAPE and cold-start scale; kept to measure disagreement with the active mode.

Algorithm 2 AdmissionReject($S_{\min}, m$)

Require: best score $S_{\min}$, mode m , SLO L , recent e2e window $\mathcal{Y}$

```

1: if  $m = \text{rank\_only}$  then return false
2: end if
3: if  $m = \text{empirical}$  then
4:   if  $|\mathcal{Y}| < 8$  then return false ▷ warm-up: do not trust  $\hat{y}$ 
5:   end if
6:    $q \leftarrow \text{Percentile}(\mathcal{Y}, p)$ 
7:    $Q_{75} \leftarrow \text{Percentile of current feasible scores at 75\%}$ 
8:   return  $(q > L) \wedge (S_{\min} \geq Q_{75})$ 
9: end if
10: return  $(S_{\min} > L)$  ▷ absolute

```

Counters `would_reject_absolute`, `would_admit_absolute`, and `absolute_vs_active_disagree` record, on every pick, whether absolute mode would

have made a different decision; these are the primary metrics for admission safety (Section 7.6). Rejected requests return HTTP 503 with `Retry-After`.

7 Experimental evaluation

7.1 Methodology

Hardware and software. Unless noted, engines are unmodified vLLM [1] serving Qwen2.5-3B-Instruct [55] with a Hugging Face tokenizer behind the `dio-serve` gateway. Homogeneous dual-T4, controlled skew, multi-turn, admission, and multi-SKU cells use the public experiment scripts in the artifact repository (Declarations).

Experiment families (Regimes A–D; non-interchangeable). We label families for cross-reference. Lettering skips Regime B: an early single-seed Locust pilot with longer-decode traces that is not used for any claim in this paper.

1. **Regime A — matched replicas (real dual-T4):** identical peers, `max_tokens=32`, $n=10$ seeds $\times$ 30 requests (Section 7.3).
2. **Regime C — controlled end-to-end latency asymmetry:** (i) CPU multi-seed simulation $n=10$; (ii) real dual-T4 with one peer under a controlled $\times 2$ end-to-end latency inflation (delay proxy), $n=10 \times 30$ (Section 7.4).
3. **Regime D — physical heterogeneity (real T4+A30):** Tesla T4 (16 GB) and NVIDIA A30 (24 GB), unmodified vLLM, no injected delay, mixed `max_tokens`, primary cell $n=5$ seeds $\times$ 40 requests, plus an offset robustness cell and fixed-length slope-sensitivity cells (Section 7.5).
4. **Admission safety (real dual-T4; not lettered as a Regime):** $n=10 \times 40$ with a deliberately tight SLO; a calibration stress test, not a throughput comparison (Section 7.6).

Baselines and metrics. Round-Robin (RR), Least-Loaded (LL), RLS [17] ($d=2$, $\lambda=0.99$), and *Latency-EWMA* under the same engines where applicable. Latency-EWMA maintains an exponentially weighted moving average of observed end-to-end latency independently for each worker and routes to the lower current estimate (cold-start: each worker is visited at least once before pure exploitation). Unlike NLMS, Latency-EWMA does *not* condition latency on token count; it is a lightweight black-box recent-latency baseline, not a token-aware predictor. Primary metrics: p50/p99 e2e mean $\pm$ std across seeds; routing fraction to each backend; admission reject rate and completions. For each seed, p99 is computed over that seed’s request-level latency distribution; reported mean $\pm$ SD then summarizes the per-seed p99 across seeds. Because each seed contains only 30–40 sequential requests, p99 is a per-run tail indicator rather than a high-confidence stationary production tail estimate. MAPE is reported only for token-conditioned latency predictors (NLMS/RLS and shadow predictors under RR/LL), and is not used as a fairness metric for Latency-EWMA. Matched multi-node Orca [9], vLLM-distributed, and Production Stack GPU bake-offs are out of scope.

7.2 Control-plane overhead

We microbench only the gateway’s `pick()` path (worker scan, cost evaluation, prefix hash), excluding network and engine decode. On a laptop-class CPU host with 32 mock backends and 100,000 sequential scheduling decisions (no GPU, synthetic worker

delay), the median scheduling cost is approximately $14\mu\text{s}$ per request; the local simulation cell in Section 7.4 reports a similar $\sim 9\text{--}10\mu\text{s}$ `pick()` cost for two workers. Per-worker state is $\sim 256\text{B}$. Relative to multi-millisecond LLM decode, the control plane is not the bottleneck (Figure 3).

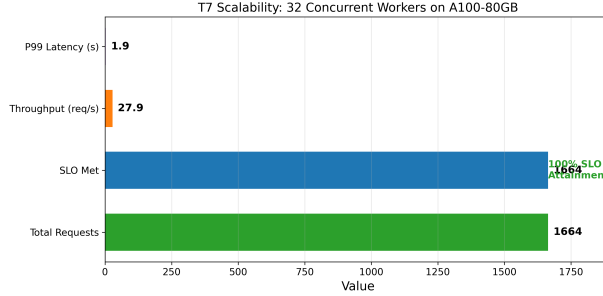


Fig. 3 Control-plane microbench (mock workers; scheduling only). Median overhead $\sim 14\mu\text{s}$; plotted e2e latency is synthetic worker delay.

7.3 Matched replicas: homogeneous dual-T4

Table 1 is the primary homogeneous multi-seed matrix. On *identical* backends, adaptive rankers stay near Round-Robin on mean p99—the expected outcome when service rates match. Secondary signals favor stability: NLMS keeps tight p99 std while Least-Loaded collapses to one worker (100% to e0); RLS can mis-route (low frac e0) without improving the tail.

NLMS MAPE on dual-T4 is high ($123 \pm 6\%$ with tokenizer). HF tokenizer vs. byte heuristic (W2, $n=3$): MAPE $117.5 \pm 10.9\%$ vs. $89.7 \pm 4.9\%$. The difference suggests that token-count estimation alone does not explain the high error; model misspecification, cold-start effects, and batching likely contribute. Routing uses $\arg \min S_w$; relative ranking under asymmetry is the primary success criterion (Regime C). Absolute admission that trusts $\hat{y}$ is therefore unsafe (Section 7.6).

Table 1 Matched-replica control. Dual-T4 + vLLM + Qwen2.5-3B, HF tokenizer, $n=10 \times 30$, `max_tokens=32`. Mean $\pm$ std. Tables 1 and 2 are independent campaigns; do not compare absolute p99 across tables. MAPE is for $\hat{y}(N)$ predictors only.

Strategy	p99 (ms)	MAPE (%)	frac e0	vs RR
NLMS	3413 ± 31	123 ± 6	0.45 ± 0.04	$-2.6 \pm 1.0\%$
RLS ($d=2$)	3466 ± 11	132 ± 6	0.12 ± 0.02	$-4.2 \pm 0.6\%$
Round-Robin	3326 ± 17	88 ± 2	0.50 ± 0.00	—
Least-Loaded	3313 ± 430	141 ± 13	1.00 ± 0.00	$+0.4 \pm 13\%$

Latency-EWMA on matched peers. A natural question is whether dual-timescale NLMS is necessary, or whether a simple recent-latency estimator would suffice when peers are equal. Table 2 reports a dual-T4 matched baseline suite ($n=10 \times 30$, `max_tokens=32`; separate campaign from Table 1, same class of hardware and load). Round-Robin remains best on p99. NLMS, RLS, and Latency-EWMA are close to

each other and slightly worse than Round-Robin—as expected when the two replicas have essentially equal service rates. Latency-EWMA distributes traffic across both peers (mean e0 fraction ≈ 0.48 , range 0.20–0.70), so it does not collapse onto a single worker when rates are matched. We do *not* claim that NLMS outperforms Latency-EWMA; this cell is a control showing that adaptive black-box rankers do not provide a latency advantage when replica service rates are effectively matched. Regimes C–D establish the ranking claim for NLMS against RR, LL, and RLS under asymmetric service; Latency-EWMA under those regimes is out of scope here.

Table 2 Lightweight black-box baselines on matched dual-T4 (Qwen2.5-3B, $n=10 \times 30$, `max_tokens=32`). Independent campaign from Table 1; absolute p99 values should not be compared across tables. Latency-EWMA is e2e-latency EMA only (no token feature). Mean $\pm$ std of seed p99; vs RR uses mean p99 relative to Round-Robin in this cell.

Strategy	p99 (ms)	frac e0	vs RR (mean p99)
Round-Robin	3311 $\pm$ 8	0.50	—
RLS ($d=2$)	3431 $\pm$ 47	—	$\approx -3.6\%$
NLMS	3461 $\pm$ 38	—	$\approx -4.5\%$
Latency-EWMA	3474 $\pm$ 54	0.48 (0.20–0.70)	$\approx -4.9\%$

7.4 Controlled end-to-end latency asymmetry

We evaluate ranking under a clear service gap in simulation and on real dual-T4 hardware. In simulation ($n=10$), with distinct true (b, s) pairs, NLMS places 97.5% of traffic on the fast worker and cuts p99 by $38.3\% \pm 2.0\%$ vs. RR (Table 3, left). RLS fails to rebalance (frac fast 0.40 ± 0.18). Local `pick()` overhead is NLMS $\approx 9.6 \mu\text{s}$ and RLS $\approx 9.4 \mu\text{s}$ at $d=2$.

On real dual-T4 ($n=10$), we apply a controlled $\times 2$ end-to-end *latency* inflation on peer e1 via an instrumented delay proxy (`latency_delay_proxy`) while both peers still run real decode on real T4 GPUs. This is a proxy for a temporarily slower peer (interference, uneven queueing, thermal pressure), not a claim that GPU service time alone is doubled. Table 4 (right): NLMS p99 3427 ± 38 ms vs. RR 6632 ± 37 ms ($48.3\% \pm 0.7\%$ improvement). RLS is weaker and noisier ($23.2\% \pm 22.9\%$). Mean frac to e0 under NLMS is 0.47 ± 0.08 . NLMS p99 stays close to the homogeneous Regime A level because the router steers work away from the slower peer; Round-Robin pays the full $\times 2$ penalty on half the traffic.

Table 3 Simulation control. Slope-skew dual workers, $n=10 \times 200$.

Metric	NLMS	RR
Frac. to fast	0.975 ± 0.00	0.50 ± 0.00
p99 (sim)	1411 ± 34	2287 ± 49
p99 vs RR	$38.3\% \pm 2.0\%$	

Table 4 Real dual-T4 control with $\times 2$ injected asymmetry, $n=10 \times 30$. NLMS p99 near Regime A reflects avoidance of the slower peer.

Strat.	p99 (ms)	frac e0	vs RR
NLMS	3427 ± 38	0.47 ± 0.08	$48.3 \pm 0.7\%$
RLS	5087 ± 1505	0.37 ± 0.19	$23.2 \pm 22.9\%$
RR	6632 ± 37	0.50 ± 0.00	—

7.5 Physical GPU heterogeneity: T4 and A30

Regime C establishes that DIO ranks correctly when a peer is *made* slow. The natural objection is that an injected $\times 2$ inflation is a proxy, whereas real hardware gaps are neither constant nor uniform across the request mix. Regime D removes the proxy: two different GPU types, no instrumented delay path, identical weights, unmodified vLLM on both. Mélange [36] addresses GPU-heterogeneous *provisioning and placement*; DIO addresses online replica selection after a heterogeneous fleet has already been deployed, learning from black-box telemetry alone without requiring an offline latency/throughput performance-capacity model before deployment.

Setup. One NVIDIA Tesla T4 (16 GB GDDR6) and one NVIDIA A30 (24 GB HBM2), each running an unmodified `vllm.entrypoints.openai.api_server` with Qwen2.5-3B-Instruct (fp16), `--gpu-memory-utilization 0.85`, in a single data-centre region (E2E Networks On-Demand). Both workers export the Prometheus `/metrics` endpoint. The DIO gateway is co-located with the T4 worker and reaches the A30 over the provider network. Because we do not measure the path-latency delta, the experiment should not be interpreted as a network-independent speedup; if anything, co-location may favour the T4. We use the per-token slope s in $\hat{y} = sN + b$ as a compact measure of the observed service-rate difference between workers (decode performance for this workload is expected to be strongly influenced by memory bandwidth, but we do not measure bandwidth directly).

Declared VRAM (not a performance profile). Each backend’s VRAM is declared explicitly (`--vram 16000 / 24000 MB`) so the soft/hard VRAM terms of the joint cost use correct device memory sizes. This is configuration of hardware memory capacity, not an offline latency or throughput profile of either SKU.

Workload design: slope identifiability. Regimes A and C fix `max_tokens`, which leaves the token feature N nearly constant. Under a near-constant regressor the decomposition of $\hat{y}$ into slope and intercept is ill-conditioned: many (s, b) pairs fit the data equally well, so the learned s is arbitrary even when $\hat{y}$ itself is accurate. This does not harm ranking — a constant absorbed into b cancels when comparing peers — but it makes any per-worker slope claim unsupportable. Regime D therefore mixes `max_tokens` $\in \{32, 64, 128, 256\}$ with varied prompt lengths, giving N genuine spread. Cell D3 (Table 8) repeats the measurement at fixed lengths to show how constant N compresses the learned slope gap.

Protocol. Sequential load (one in-flight request), $n=5$ seeds $\times$ 40 requests per strategy, fresh gateway process per seed. Strategies: dual-timescale NLMS, $d=2$ RLS, Round-Robin, Least-Loaded. Primary metrics: p50/p95/p99 e2e mean $\pm$ std across seeds, routing fraction to the A30, and paired mean p99 improvement vs. RR (wins/ n). Sequential single-in-flight load isolates ranking from continuous-batching interactions; concurrent multi-in-flight Regime D is left as future work (Section 9). Direct no-gateway probes on the same engines give a mean *end-to-end* latency ratio T4/A30 $\approx 2.68\times$ on fixed 64-token chats; this is a workload-level e2e ratio, distinct from the per-token slope ratio reported below.

Slope validation. For every worker we fit an offline least-squares model to the same $(N, \text{observed latency})$ pairs the online learner trained on. The online NLMS slope is then reported against that OLS fit. Because NLMS is a tracking filter rather

than an unbiased estimator, its slope is expected to be shrunk toward zero relative to OLS; ranking only needs order and relative costs.

Online routing results. Table 5 reports the multi-seed matrix. Both NLMS and RLS improve mean p99 over Round-Robin, with **5/5** paired wins: NLMS achieves **33.3%** mean improvement (SD 17.4%) versus 33.7% (SD 24.1%) for RLS. An exact one-sided sign test gives $p=0.031$ for the five NLMS wins, although the small sample limits statistical power. A seed-matched Wilcoxon signed-rank test on per-seed p99 (NLMS vs. RLS; $n=5$) yields $W=7$ (exact two-sided $p=1.00$), so we do not claim that NLMS is more accurate than $d=2$ RLS on mean p99. NLMS routes **89.5% $\pm$ 2.1%** of requests to the empirically lower-latency A30, with lower seed-to-seed variability than RLS. RLS is more variable: four seeds strongly prefer the A30, while one reverses the preference. NLMS also has lower p99 dispersion (SD 1328 vs. 1821 ms). Both learners use the same linear feature model; NLMS updates are constant-time for this two-parameter model, whereas generic RLS uses matrix operations. RR remains a 50/50 control. Least-Loaded routes all traffic to the T4 under one-in-flight load, where queue depth provides little differentiation; its mean p99 is 22.7% (SD 3.1%) worse than RR (0/5 wins).

Offset robustness seeds. Table 6 reports five additional NLMS/RLS seeds on the same engines (sequential mixed-length protocol; no Round-Robin). NLMS routes $92\% \pm 1.9\%$ of traffic to the A30 and RLS $97\% \pm 1.0\%$, consistent with Table 5.

Ordering check. Table 7 compares online NLMS slopes to offline OLS on the same observed pairs (post-hoc consistency check). OLS recovers a T4/A30 *per-token slope* ratio of $29.80/10.58 \approx 2.82\times$ ($R^2=0.982$ T4, 0.988 A30). The $2.68\times$ fixed-length end-to-end probe and the $2.82\times$ mixed-length slope ratio are different quantities and need not match. NLMS slopes shrink toward zero relative to OLS but preserve order (A30/T4 0.318 vs. OLS 0.355).

Fixed-length slope sensitivity (cell D3). Table 8 fixes `max_tokens` at 32, 128, or 256 ($n=3$ seeds each). NLMS still routes most traffic to the empirically lower-latency A30, but the learned T4/A30 slope ratio collapses toward 1 as N becomes nearly constant ($1.34\times$, $1.16\times$, $1.09\times$), whereas mixed-length OLS in Table 7 separates the workers by $\approx 2.82\times$. Ranking can succeed under fixed length; reliable identification of per-worker slope differences requires sufficient variation in N .

Takeaway. Under sequential multi-SKU load, black-box online ranking substantially reduces mean p99 relative to Round-Robin without an offline latency/throughput profile. NLMS and RLS have similar mean p99; NLMS shows lower routing-fraction variability toward the empirically lower-latency A30.

7.6 Admission safety under poor absolute calibration

Same dual-T4 pair; sequential short chat probes (40/seed, $n=10$). Tight SLO $L=800$ ms (below typical e2e on this setup) exercises admission under unreliable absolute $\hat{g}$ (Section 7.3). Modes: `absolute`, `empirical`, `rank_only`.

Table 9 and Figure 4: `absolute` rejects **0.40 $\pm$ 0.21** of requests across seeds (HTTP 503). `empirical` and `rank_only` reject **0%** and complete all 40 requests per seed. Completed-request p99 is similar across modes (~ 3.4 s). The relevant metric is reject *rate*: absolute mode over-rejects under high MAPE, while empirical and rank_only do not [21, 33, 51].

Table 5 Online routing on real T4+A30 unmodified vLLM (Qwen2.5-3B, mixed `max_tokens`, $n=5$ seeds $\times$ 40 requests, no injected delay, sequential). Mean $\pm$ SD across seeds; vs RR is paired mean p99 improvement (wins/ n); frac A30 is the routing fraction to the empirically lower-latency GPU under this workload.

Strategy	p50 (ms)	p99 (ms)	frac A30	vs RR (p99)
NLMS	1130 $\pm$ 271	5092 $\pm$ 1328	0.895 $\pm$ 0.021	33.3 $\pm$ 17.4% (5/5)
RLS	1183 $\pm$ 490	5045 $\pm$ 1821	0.795 $\pm$ 0.375	33.7 $\pm$ 24.1% (5/5)
Round-Robin	1524 $\pm$ 222	7629 $\pm$ 210	0.50 $\pm$ 0.00	—
Least-Loaded	3265 $\pm$ 1160	9356 $\pm$ 257	0.00 $\pm$ 0.00	-22.7 $\pm$ 3.1% (0/5)

Table 6 Offset robustness cell on the same T4+A30 pair ($n=5$ additional seeds $\times$ 40 requests, sequential mixed `max_tokens`; NLMS/RLS only). Mean $\pm$ SD across seeds.

Strategy	p99 (ms)	frac A30
NLMS	5555 $\pm$ 1142	0.920 $\pm$ 0.019
RLS	4562 $\pm$ 1067	0.970 $\pm$ 0.010

Table 7 Ordering check on T4+A30. Online NLMS per-token slope vs. offline OLS on the same observed (N , latency) pairs under RR traffic ($n=5$ seeds; R^2 from the pooled OLS fit). Ratio is A30/T4 (smaller = lower per-token cost on A30).

Worker	VRAM (MB)	NLMS s	OLS s	R^2
T4	16000	16.03 $\pm$ 1.67	29.80	0.982
A30	24000	5.10 $\pm$ 0.57	10.58	0.988
Ratio (A30/T4)	—	0.318	0.355	—

Table 8 Cell D3: fixed `max_tokens` on T4+A30 (NLMS only, $n=3$ seeds each). Learned slopes lose separation as N becomes nearly constant; routing to A30 remains high.

<code>max_tokens</code>	p99 (ms)	frac A30	NLMS s_{T4}	NLMS s_{A30}
32	728 $\pm$ 16	0.975	2.68	2.00
128	3621 $\pm$ 5	0.900	7.79	6.74
256	7184 $\pm$ 4	0.892	9.42	8.64

8 Positioning against existing systems

Published systems use different hardware, fleets, models, and optimization layers; their numbers are not a leaderboard against ours. DIO’s contribution is a reproducible measurement of black-box online ranking on two unequal unmodified vLLM replicas, plus the admission-calibration boundary. Both DIO and the Production Stack router speak OpenAI-compatible HTTP; Production Stack also supports static discovery

Table 9 Real dual-T4 admission suite ($n=10 \times 40$; SLO $L=800$ ms). Mean $\pm$ std. p99 is over completed requests only.

Mode	Reject rate	Completed / 40	p99 completed (ms)
absolute	0.40 ± 0.21	24.2	3415 ± 63
empirical	0.00 ± 0.00	40	3407 ± 43
rank_only	0.00 ± 0.00	40	3409 ± 57

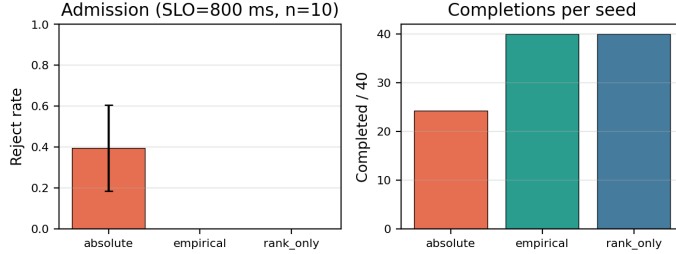


Fig. 4 Dual-T4 admission under tight SLO ($n=10$). Absolute rejects $\sim 40\%$; empirical and rank_only reject none.

Table 10 Scope comparison. This is not a performance leaderboard: published systems use different models, clusters, workloads, and metrics.

System	Interface to engines	Primary routing signal	Relation to DIO
vLLM Production Stack [42]	Kubernetes or static backends	Session/prefix-aware policies	Deployable without a full cluster (static discovery).
Gateway Inference Extension / llm-d [38, 39]	API Kubernetes End-point Picker	Engine metrics, request cost, prefix/cache state	Broader cluster integration; DIO is a smaller standalone HTTP gateway.
NVIDIA Dynamo [41]	Dynamic worker discovery and KV events	KV overlap plus prefill/decode load	Richer cache state; DIO uses ordinary HTTP endpoints and e2e telemetry.
Llumnix [14]	Engine hooks plus live migration	Fine-grained instance state and rescheduling	Co-designed system with a broader optimization scope.
DIO (this work)	Unmodified OpenAI-compatible APIs	Online per-worker latency ranking	T4+A30 routing without a pre-deployment latency/throughput model.

without Kubernetes, so operators can extend from a laptop smoke path to a cluster-native install without changing the engine interface.

9 Discussion and limitations

What the experiments establish. NLMS ranking helps when workers have an observed service gap, either controlled (Section 7.4) or physical (Section 7.5); it should not be expected to improve matched replicas (Section 7.3). Prefer **empirical** or **rank_only** admission when absolute prediction error is known to be high (Section 7.6). Optional affinity and scraped-metric paths are implementation capabilities and are not sold as independent performance wins here.

Limitations. (i) **Physical seed count and coverage.** The physical primary cell uses five paired seeds ($\times 40$ requests) plus an offset five-seed routing-robustness cell on one T4+A30 pair, one model (Qwen2.5-3B), and one cloud region. Small n and single-model/SKU coverage limit generality; p99 is outlier-sensitive, so we report mean $\pm$ SD, paired win counts, and the offset routing check rather than claiming broad robustness from five seeds alone. (ii) **Sequential load.** Regime D is deliberately one-in-flight, which isolates ranking from continuous batching but removes concurrent queue/KV dynamics typical of production multi-instance serving. Concurrent multi-in-flight heterogeneous ranking is not measured here. (iii) **Baselines.** Latency-EWMA is reported only on matched dual-T4 (Section 7.3), so we do not claim NLMS over EWMA under skew or multi-SKU load. On Regime D, NLMS and RLS have similar mean p99; NLMS has lower routing-fraction variability and constant-time two-parameter updates. (iv) **Other.** Gateway co-location with the T4 may favour that peer on the network path. Direct probes and offline OLS are post-hoc checks, not controller inputs. The controlled dual-T4 delay is a mechanism study, not a second physical SKU pair. Coefficients are configuration defaults without a grid search. We do not report a shared-workload GPU bake-off against production routers (Table 10).

10 Conclusion

DIO is a thin OpenAI-compatible gateway for multi-instance unmodified vLLM deployments. On matched dual-T4 peers, adaptive rankers including Latency-EWMA stay near Round-Robin. Under controlled latency asymmetry and a physical T4+A30 pair, dual-timescale NLMS-style ranking learns a useful relative ordering from black-box end-to-end telemetry without an offline latency/throughput profile. On the physical cell, NLMS and RLS have similar mean p99; NLMS has lower routing-fraction variability and constant-time two-parameter updates. The same model is too poorly calibrated for absolute admission (MAPE ~ 90 – 130% ; reject rate 0.40 ± 0.21). Ranking and admission should be separate policies when predictive latency estimates are poorly calibrated. Source code and artifacts are public.

Acknowledgements. We thank colleagues for feedback on earlier drafts.

Appendix A Notation summary

	Symbol	Meaning
Main symbols.	$s^{\text{fast}}, s^{\text{slow}}, s^{\text{eff}}$	Dual-timescale slopes (ms/token)
	b_w	Intercept (ms)
	$N, y, \hat{y}$	Tokens; actual / predicted e2e latency (ms)
	S_w	Joint routing cost for worker w
	Q_w, B	Pending depth; batch amortization
	KV_w, W_w, Hit_w	Scraped KV usage, waiting count, prefix hit rate
	c_{kv}, c_q, c_p	Hybrid cost coefficients (ms scale)
	C	Prefix affinity bonus (ms)
	L	Latency SLO (ms)

Statements and Declarations

Funding. The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Competing Interests. The authors have no relevant financial or non-financial interests to disclose.

Data Availability. The experimental data generated and analysed during the current study, including aggregated multi-seed experimental results, are publicly available in the DIO repository: <https://github.com/nisaral/DIO>. The repository also contains the source code, experimental scripts, configurations, analysis materials, and paper artifacts required to reproduce the reported experiments.

Code Availability. Source code for `dio-serve` and the experiment harnesses used in this paper is available at <https://github.com/nisaral/DIO>.

Ethics approval and consent to participate. Not applicable (no human subjects or personal data).

Consent for publication. Not applicable.

Materials availability. Not applicable.

References

- [1] Kwon, W., *et al.*: Efficient memory management for large language model serving with PagedAttention. In: SOSP (2023)
- [2] Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C.H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J.E., Barrett, C., Sheng, Y.: SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104; see also NeurIPS 2024 version [4] (2023)
- [3] Hugging Face: Text Generation Inference. <https://github.com/huggingface/text-generation-inference> (2023)
- [4] Zheng, L., *et al.*: SGLang: Efficient execution of structured language model programs. In: NeurIPS (2024). RadixAttention prefix cache
- [5] F5, Inc.: NGINX HTTP Load Balancer. <https://nginx.org/> (2024)
- [6] Cloud Native Computing Foundation: Kubernetes. <https://kubernetes.io/> (2024)
- [7] Envoy Project Authors: Envoy Proxy. <https://www.envoyproxy.io/> (2024)
- [8] Ray Project: Ray Serve. <https://docs.ray.io/en/latest/serve/> (2024)
- [9] Yu, G.-I., *et al.*: Orca: A distributed serving system for Transformer-based generative models. In: OSDI (2022)

- [10] Agrawal, A., *et al.*: Sarathi-Serve: Efficiently serving large language models via hybrid scheduling. In: OSDI (2024)
- [11] Eisenbud, D.E., *et al.*: Maglev: A fast and reliable software network load balancer. In: NSDI (2016)
- [12] Dean, J., Barroso, L.A.: The tail at scale. *Communications of the ACM* **56**(2) (2013)
- [13] Zhang, Y., Chen, Y., Mo, X., Xi, A., Li, J., Wu, W.: A Predictive and Synergistic Two-Layer Scheduling Framework for LLM Serving (NexusSched). arXiv:2509.23384 (2025)
- [14] Wu, B., *et al.*: Llumnix: Dynamic Scheduling for Large Language Model Serving (2024)
- [15] Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., Jin, X.: LoongServe: Efficiently serving long-context large language models with elastic sequence parallelism. In: SOSPP (2024)
- [16] OpenAI: OpenAI API Reference. <https://platform.openai.com/docs/api-reference> (2024)
- [17] Haykin, S.: Adaptive Filter Theory, 4th edn. Prentice Hall, Upper Saddle River, NJ (2002)
- [18] Prometheus Authors: Prometheus: Monitoring System and Time Series Database. <https://prometheus.io/> (2024)
- [19] Prabhu, R., *et al.*: vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention (2024)
- [20] Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., Jin, X.: Fast Distributed Inference Serving for Large Language Models. arXiv:2305.05920 (2023)
- [21] Zhong, Y., *et al.*: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: OSDI (2024)
- [22] Patel, P., *et al.*: Splitwise: Efficient Generative LLM Inference Using Phase Splitting. arXiv preprint (2024)
- [23] Qin, R., Li, Z., He, W., Zhang, M., Wu, Y., Zheng, W., Xu, X.: Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving (2024)
- [24] Hu, C., Huang, H., Hu, J., Xu, J., Chen, X., Xie, T., Wang, C., Wang, S., Bao, Y., Sun, N., Shan, Y.: MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool (2024)

- [25] Gim, I., et al.: Prompt Cache: Modular Attention Reuse for Low-Latency Inference (2023)
- [26] Juravsky, J., et al.: Hydragen: High-Throughput LLM Inference with Shared Prefixes (2024)
- [27] Yao, J., et al.: CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion (2024)
- [28] Cheng, Y., et al.: LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv / open-source cache layer (2024)
- [29] Srivatsa, V., et al.: Preble: Efficient Distributed Prompt Scheduling for LLM Serving (2024)
- [30] NVIDIA Corporation: Triton Inference Server (2024)
- [31] Crankshaw, D., et al.: Clipper: A low-latency online prediction serving system. In: NSDI (2017)
- [32] Romero, F., et al.: INFaaS: Automated model-less inference serving. In: USENIX ATC (2021)
- [33] Gujarati, A., et al.: Serving DNNs like clockwork: Performance predictability from the bottom up. In: OSDI (2020)
- [34] Li, Z., et al.: AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In: OSDI (2023)
- [35] Wu, J., et al.: Unlock the Potential of Fine-grained LLM Serving via Module-wise Autoscaling (CoCoServe) (2025)
- [36] Griggs, T., Liu, X., Yu, J., Kim, D., Chiang, W.-L., Cheung, A., Stoica, I.: Mélange: Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity. arXiv:2404.14527 (2024)
- [37] Fu, Y., et al.: ServerlessLLM: Low-Latency Serverless Inference for Large Language Models (2024)
- [38] Kubernetes SIG-Network: Gateway API Inference Extension. <https://github.com/kubernetes-sigs/gateway-api-inference-extension> (2025)
- [39] Google Cloud: llm-d: LLM-aware Gateway for Kubernetes. <https://github.com/GoogleCloudPlatform/llm-d> (2025)
- [40] Google Cloud: GKE Inference Gateway. <https://cloud.google.com/kubernetes-engine/docs/concepts/inference-gateway> (2025)

- [41] NVIDIA Corporation: NVIDIA Dynamo: Distributed Inference Serving System. <https://github.com/nvidia/dynamo> (2025)
- [42] vLLM Project: vLLM Production Stack — Prefix-aware, KV-aware, and Session Routing. <https://docs.vllm.ai/projects/production-stack/en/latest/> (2025)
- [43] Alibaba Cloud: ACK/ACS Inference Gateway with Prefix-Cache-Aware Routing. <https://www.alibabacloud.com/help/en/ack/> (2025)
- [44] SGLang Project: SGLang Router — Cache-Aware Load Balancing. <https://github.com/sgl-project/sglang> (2025)
- [45] Envoy AI Gateway Authors: Envoy AI Gateway. <https://github.com/envoyproxy/ai-gateway> (2025)
- [46] Sayed, A.H.: Adaptive Filters. Wiley-IEEE Press, Hoboken, NJ (2008)
- [47] Little, J.D.C.: A proof for the queuing formula: $L = \lambda W$. Operations Research (1961)
- [48] Kleinrock, L.: Queueing Systems, Volume I: Theory. Wiley, New York (1975)
- [49] Coffman, E.G., Denning, P.J.: Operating Systems Theory. Prentice-Hall, Englewood Cliffs, NJ (1973)
- [50] Williams, S., Waterman, A., Patterson, D.: Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM **52**(4) (2009)
- [51] Wang, Y., Chen, Y., Li, Z., Tang, Z., Guo, R., Wang, X., Wang, Q., Zhou, A.C., Chu, X.: Towards Efficient and Reliable LLM Serving: A Real-World Workload Study. arXiv:2401.17644; also released as BurstGPT workload study (2024)
- [52] Agrawal, A., Kedia, N., Mohan, J., Panwar, A., Kwatra, N., Gulavani, B.S., Ramjee, R., Tumanov, A.: Vidur: A Large-Scale Simulation Framework for LLM Inference. arXiv:2405.05465 (2024)
- [53] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D., Wang, C.: AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 (2023)
- [54] LangChain: LangGraph: Build Resilient Language Agents as Graphs. <https://github.com/langchain-ai/langgraph> (2024)
- [55] Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., Li, C., Li, C., Liu, D., Huang, F., Dong, G., Wei, H., Lin, H., Tang, J., Wang, J., Yang, J., Tu, J., Zhang, J., Ma, J., Yang, J., Xu, J., Zhou, J., Lin, J., et al.: Qwen2.5 Technical